

Project Proposal: High-Performance N-Body Simulation of Interacting Galaxies Using CUDA

GPU Programming Project

December 2025

Project Goal

The objective of this project is to implement a GPU-accelerated N-body simulation of two colliding spiral galaxies. This scenario provides a “democratic” system where thousands of bodies interact with equal weight, making it an ideal benchmark for evaluating GPU memory hierarchy management. We will treat this primarily as a systems optimization problem, focusing on how CUDA-specific design choices affect throughput and scalability on an NVIDIA RTX 3060.

Simulation Scenario & Scale

- **Setting:** A collision between two rotating star disks (Antennae-like). This is computationally more demanding than a solar system model due to the lack of a single dominant mass, requiring a full $O(N^2)$ pairwise force evaluation.
- **Scale:** We will target a range of $N = 8,192$ to $32,768$ star particles. At $N = 32,768$, the system must process over 1 billion interactions per frame.
- **Scalability Testing:** We will measure performance across power-of-two increments (2^{10} to 2^{15}) to identify the point at which the RTX 3060 hardware reaches saturation.

Interaction Model & Numerical Method

Physics Model

We will utilize Newtonian gravity with a softening parameter ε to prevent numerical singularities and “ejections” during close encounters:

$$\mathbf{F}_i = \sum_{j \neq i}^N G \frac{m_i m_j}{(\|\mathbf{r}_j - \mathbf{r}_i\|^2 + \varepsilon^2)^{3/2}} (\mathbf{r}_j - \mathbf{r}_i)$$

Integration

To ensure long-term stability and energy conservation of the galactic structures, we will implement the **Velocity Verlet** integrator (a symplectic method):

1. $\mathbf{v}(t + \frac{1}{2}\Delta t) = \mathbf{v}(t) + \frac{1}{2}\mathbf{a}(t)\Delta t$
2. $\mathbf{r}(t + \Delta t) = \mathbf{r}(t) + \mathbf{v}(t + \frac{1}{2}\Delta t)\Delta t$

3. $\mathbf{a}(t + \Delta t) = \text{Force}(\mathbf{r}(t + \Delta t))$
4. $\mathbf{v}(t + \Delta t) = \mathbf{v}(t + \frac{1}{2}\Delta t) + \frac{1}{2}\mathbf{a}(t + \Delta t)\Delta t$

CUDA Parallelization & Kernel Design

Our implementation will focus on three key GPU optimization strategies:

- **Shared Memory Tiling:** We will develop a kernel where thread blocks collaboratively load tiles of particle data into shared memory. This is expected to reduce Global Memory traffic by a factor proportional to the block size (e.g., 256x).
- **Memory Coalescing via SoA:** Particle data (position and mass) will be stored in a Structure-of-Arrays (SoA) format using `float4` alignment to maximize the efficiency of the L2 cache and DRAM bus.
- **Thread Mapping:** We will assign one thread per body, with force accumulation happening entirely in registers to minimize global memory writes.

Expected Bottlenecks & Mitigations

- **Memory Bandwidth:** The naive implementation will likely be stalled by DRAM latency. *Mitigation:* Tiling in Shared Memory.
- **Instruction Throughput:** With tiling, the kernel will become compute-bound. *Mitigation:* We will investigate loop unrolling and the use of fast math intrinsics (`__fdividef`, etc.).
- **Host-Device Transfer:** Moving 32k particles to the CPU for rendering is a bottleneck. *Mitigation:* Use CUDA-OpenGL Interop to render directly from GPU buffers.

Evaluation & Metrics

- **Correctness:** Tracking total energy ($K + U$) drift and visually verifying the formation of tidal tails.
- **Performance:** Reporting Giga-Interactions Per Second (GIPS) and execution time per step.
- **Profiling:** Using Nsight Compute to report achieved occupancy and SM throughput to justify kernel tuning decisions.